
# Trending Artists Pipeline with Reddit, Airflow, and AWS

08.14.2024

—

Aman Thakur
University of Massachusetts
285 Old Westport Rd, Dartmouth MA 02747

# 1. Introduction

## Project Overview

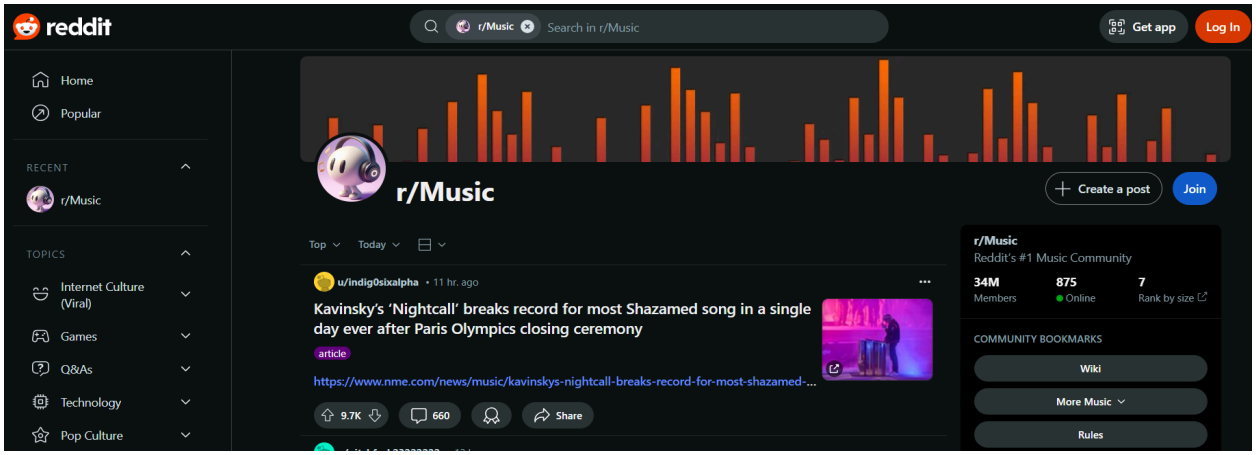
In the rapidly evolving music industry, understanding trends and artist popularity is crucial. This project aims to leverage Reddit data to create a dynamic dashboard that tracks and visualizes trending music topics and artists from the subreddit r/music. By tapping into Reddit's vast user-generated content, this project provides a real-time view of music trends, offering valuable insights for industry stakeholders.

## Objective

- Extract top Reddit posts from r/music to gather current music trends.
- Transform and clean the extracted data to align with artist information.
- Store the processed data in AWS S3 and update it regularly.
- Visualize the data on an Amazon QuickSight dashboard to display trending artists, genres, and other metrics.
- Automate the entire pipeline using Apache Airflow for seamless data updates and visualization.

## Significance

This project provides significant value by offering real-time insights into music trends and artist popularity. It supports data-driven decision-making for music industry professionals, enables artists to gauge their public perception, and helps fans discover emerging trends in the music world.

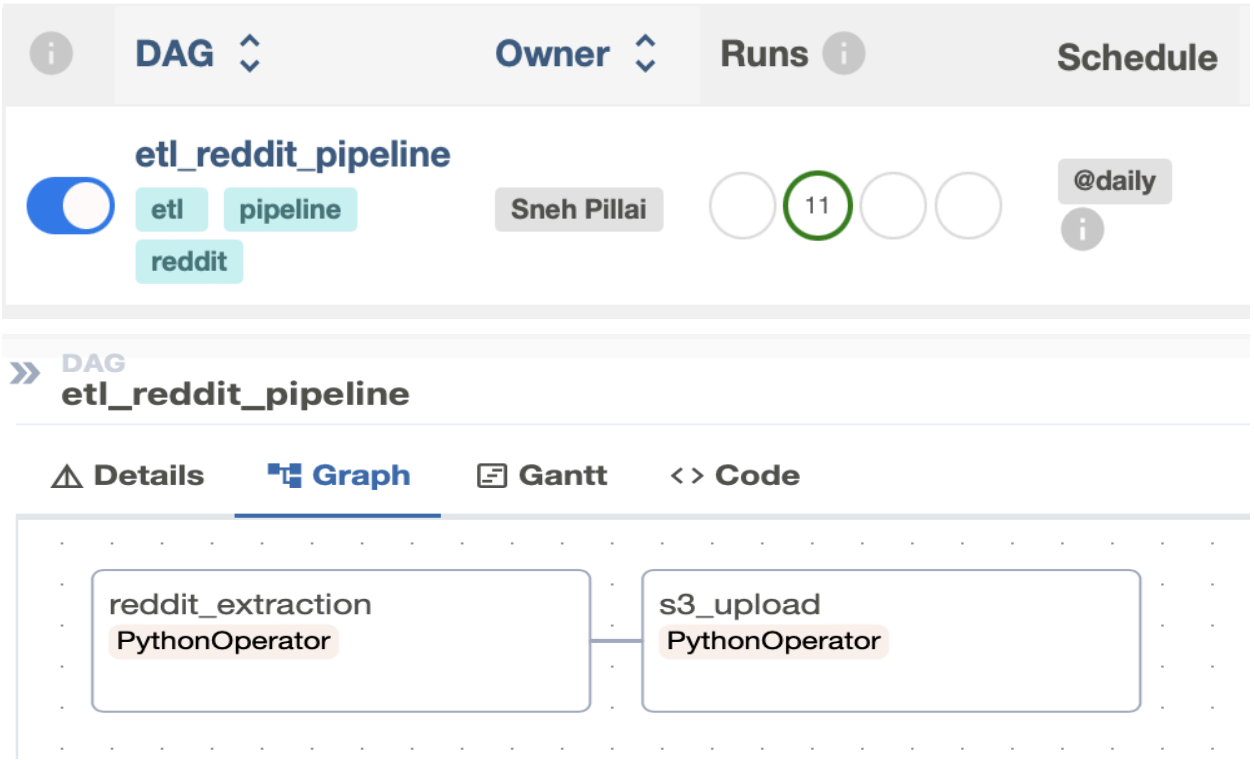



*This screenshot of the r/music subreddit showcases the community where the data is sourced. The subreddit is a hub for music enthusiasts to discuss the latest trends, share their favorite tracks, and discover new artists. The rich and diverse user-generated content here forms the foundation for the insights presented in the project.*

# 2. Technologies Used

## Apache Airflow:

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows. In this project, Airflow is utilized to orchestrate the various tasks within the data pipeline, ensuring that each step—from data extraction to transformation and storage—is executed in the correct sequence. The platform's Directed Acyclic Graphs (DAGs) provide a clear visual representation of the workflow, making it easier to monitor and manage the entire process. Airflow's robust scheduling capabilities and ability to handle complex dependencies make it an essential tool in maintaining the reliability of the pipeline.




*The screenshot showcases the Apache Airflow DAG (Directed Acyclic Graph), illustrating how the pipeline tasks are orchestrated. Each task is a crucial step in the data processing flow, ensuring smooth and timely updates to the dashboard.*

## Docker:

Docker is a platform that enables developers to automate the deployment of applications inside lightweight, portable containers. These containers package the application and its dependencies, ensuring consistent performance across different environments. In this project,

Docker is used to containerize the Python scripts and other components of the pipeline, making them easily deployable across different environments. Docker's ability to create isolated environments means that each component can run independently, reducing conflicts and ensuring smooth integration within the broader pipeline.

## Containers

Give feedback

Container CPU usage
298.74% / 300% (3 CPUs available)

Container memory usage
646.24MB / 3.74GB

Show charts

Search

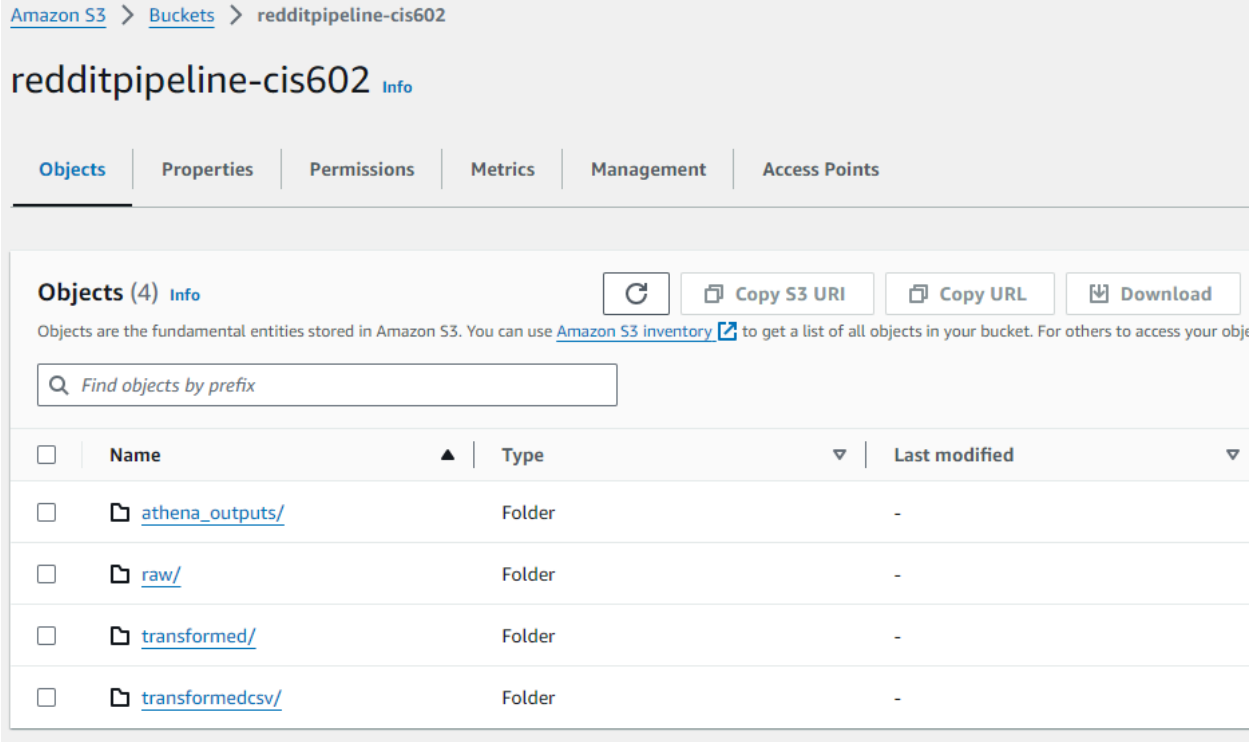
Only show running containers

| Name | Image | Status | Port(s) | CPU (%) | Last started | Actions |
|---|---|---|---|---|---|---|
| airflow-webserver-1<br>c01ca79ad40a | custom-airflow:2.7.<br>python3.9 | Running | 8080:8080 | 90.99% | 13 seconds ago | |
| airflow-scheduler-1<br>212ea025b105 | custom-airflow:2.7.<br>python3.9 | Running | | 55.92% | 13 seconds ago | |
| airflow-worker-1<br>af3bc4597bc8 | custom-airflow:2.7.<br>python3.9 | Running | | 63.79% | 13 seconds ago | |
| airflow-init-1<br>27ab838c01a1 | custom-airflow:2.7.<br>python3.9 | Running | | 85.76% | 13 seconds ago | |
| postgres-1<br>2858eaa4f06e | postgres:12 | Running | 5433:5433 | 1.75% | 13 seconds ago | |
| redis-1<br>010882c75e97 | redis:latest | Running | 6379:6379 | 0.53% | 13 seconds ago | |

```yaml
services:
  postgres:
    image: postgres:12
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: airflow_reddit
      POSTGRES_PORT: 5433
    ports:
      - "5433:5433"

  redis:
    image: redis:latest
    ports:
      - "6379:6379"

  airflow-init:
    <<: *airflow-common
    command: >
      bash -c "pip install -r /opt/airflow/requirements.txt && airflow db init && airflow db upgrade && airflow use
    restart: "no"

  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080"

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler

  airflow-worker:
    <<: *airflow-common
    command: celery worker
```

## AWS S3

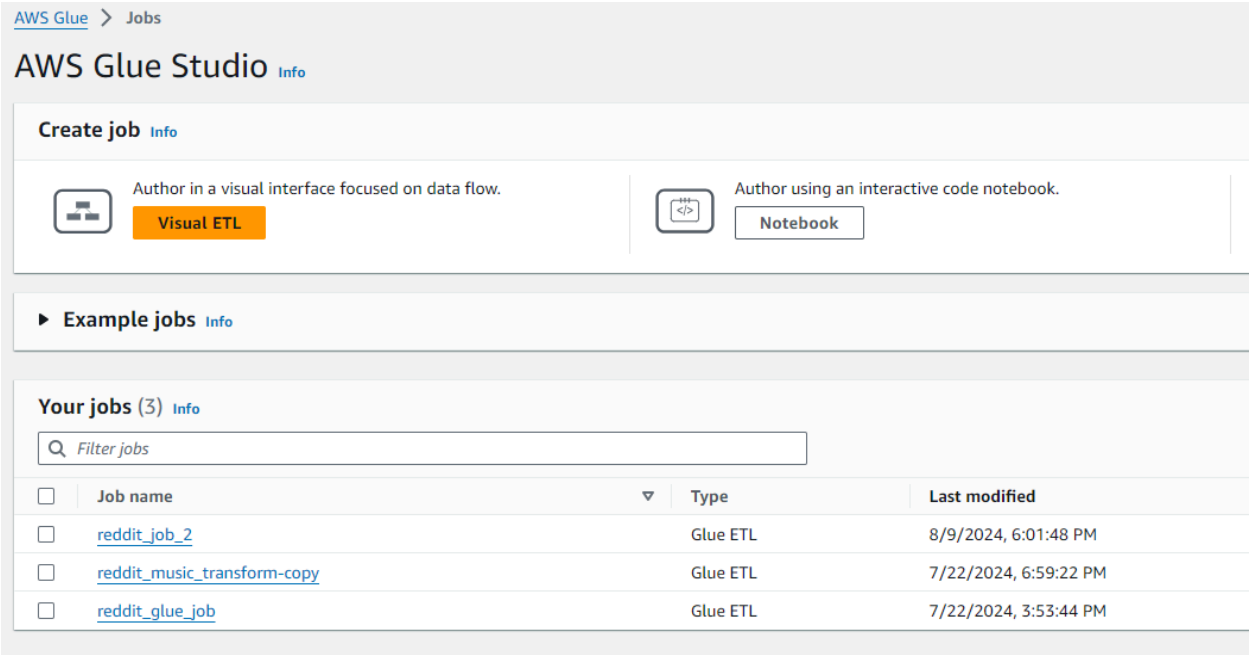
Amazon S3 is a scalable, high-speed, and cost-effective storage service designed for large-scale data storage. In this project, AWS S3 serves as the central repository where all data extracted from Reddit is stored. S3's ability to handle both structured and unstructured data, along with its high durability and availability, ensures that the data remains secure and accessible throughout the pipeline. It also plays a crucial role in the staging and transformation phases, providing an efficient way to store and retrieve intermediate datasets.




*This screenshot shows the S3 bucket structure used to store the raw and processed data extracted from Reddit. The bucket is organized to facilitate efficient data storage and retrieval.*

## AWS Glue

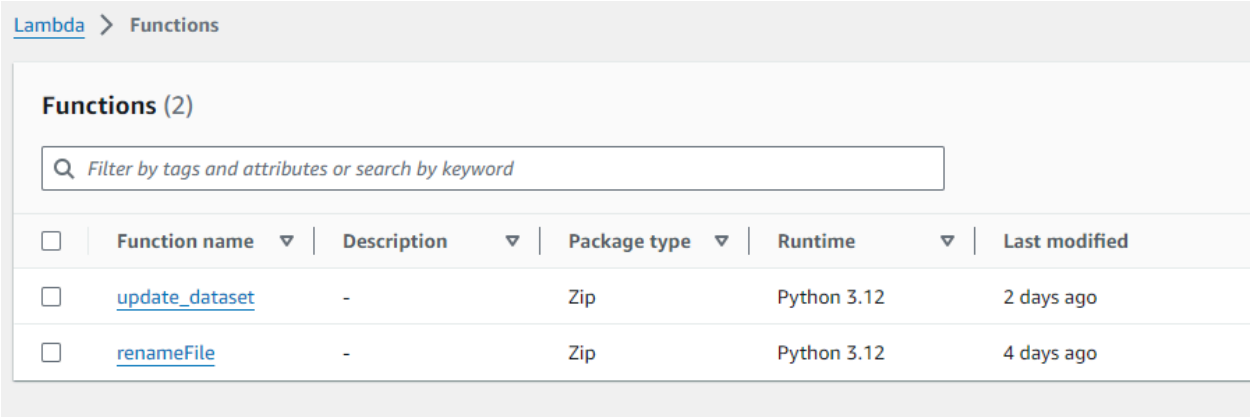
AWS Glue is a fully managed ETL (Extract, Transform, Load) service that automates the process of data preparation for analysis. In this project, AWS Glue is leveraged to clean and transform the raw data obtained from Reddit, converting it into a format suitable for deeper analysis. Glue's serverless nature allows for automatic scaling, handling varying workloads without the need for manual intervention. By integrating with other AWS services like S3 and Athena, Glue plays a pivotal role in streamlining data processing workflows.

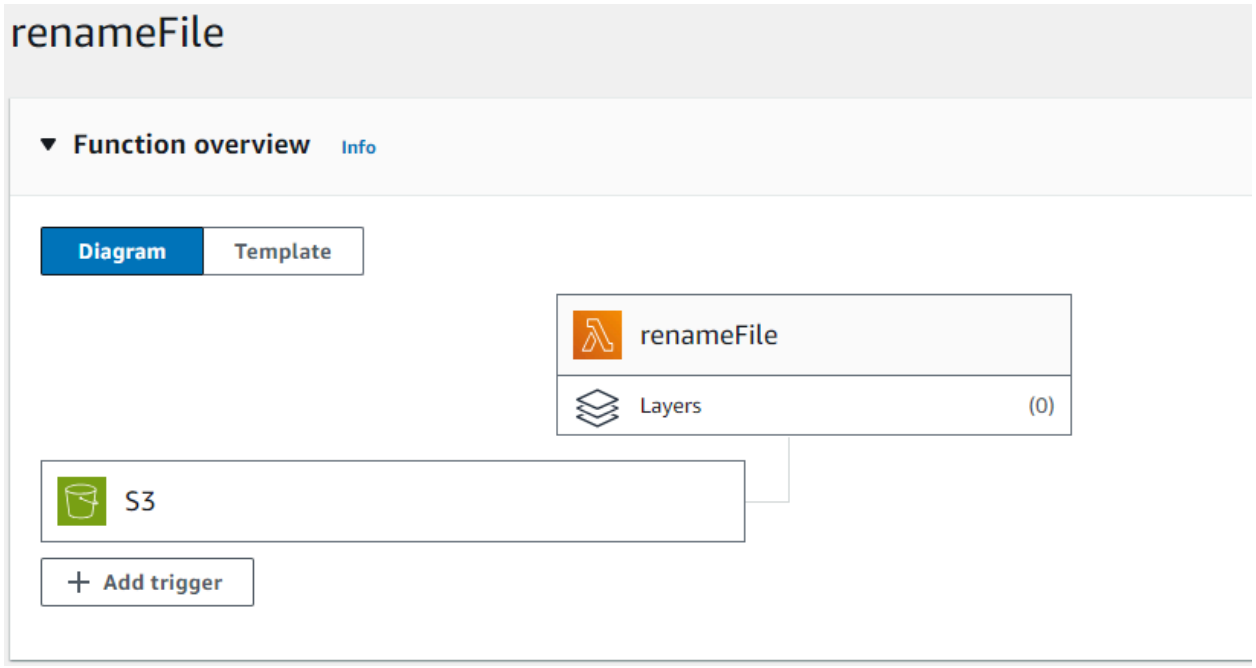


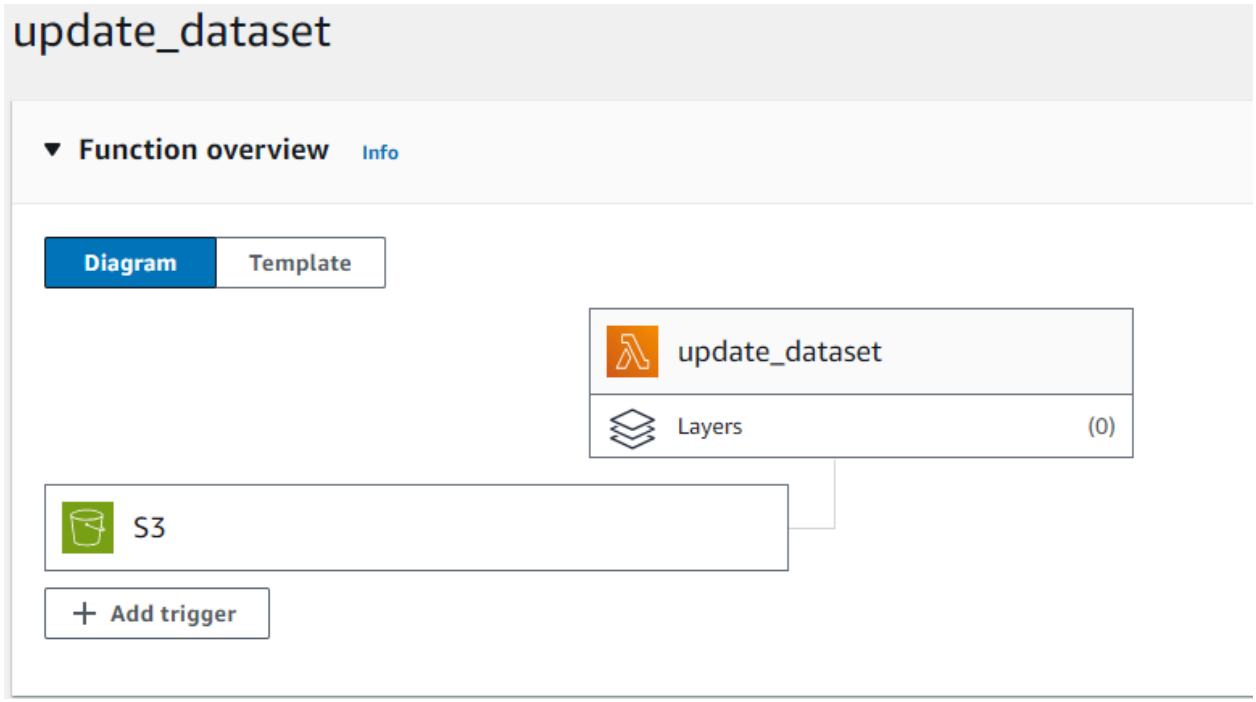

*Here, the AWS Glue jobs are depicted, which automate the transformation of raw Reddit data into a structured format ready for analysis. The jobs are configured to extract key information, such as artist names and genres, from the unstructured Reddit posts.*

## AWS Lambda

AWS Lambda is a serverless compute service that lets you run code without provisioning or managing servers. In this project, Lambda functions are used to automate various tasks within the data pipeline, such as triggering the execution of the Python scripts or initiating data processing jobs. The serverless nature of Lambda allows for automatic scaling in response to demand, ensuring that the pipeline can handle varying workloads efficiently. Lambda's ability to integrate with other AWS services, such as S3 and Glue, makes it a powerful tool for building responsive and scalable data workflows.



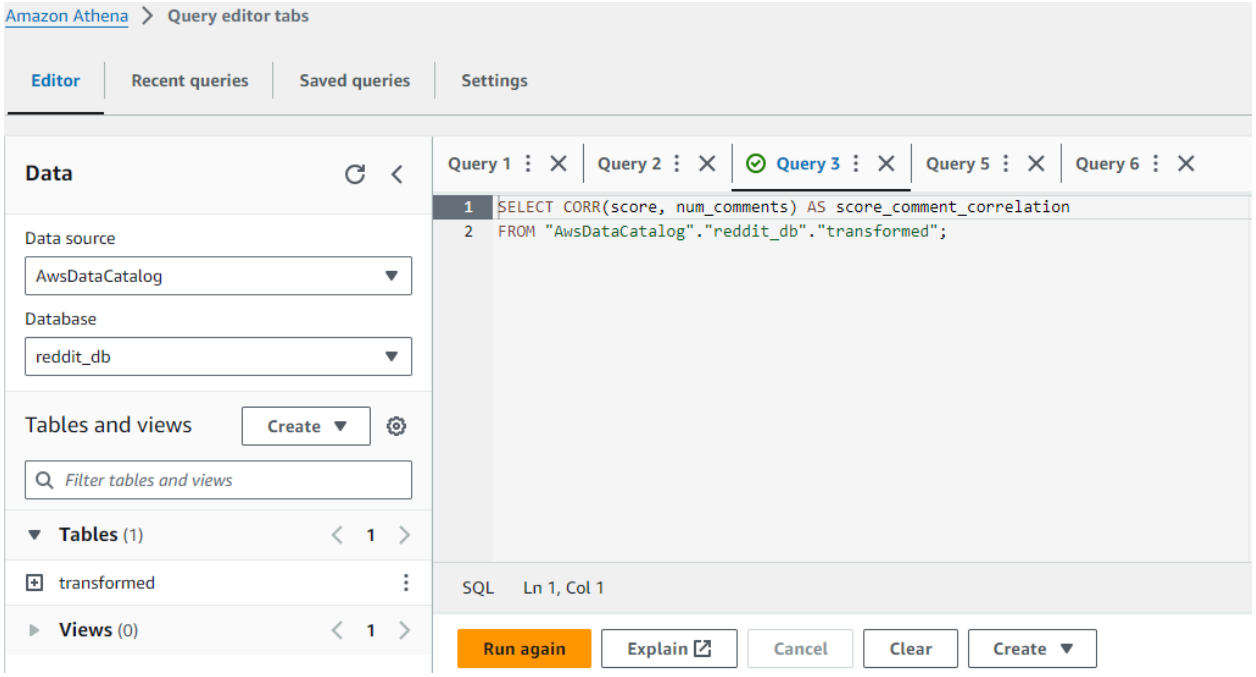



*The screenshot displays the AWS Lambda functions responsible for refreshing the datasets in Amazon QuickSight. This automation ensures that the dashboard is always up-to-date with the latest Reddit data.*

## Amazon Athena

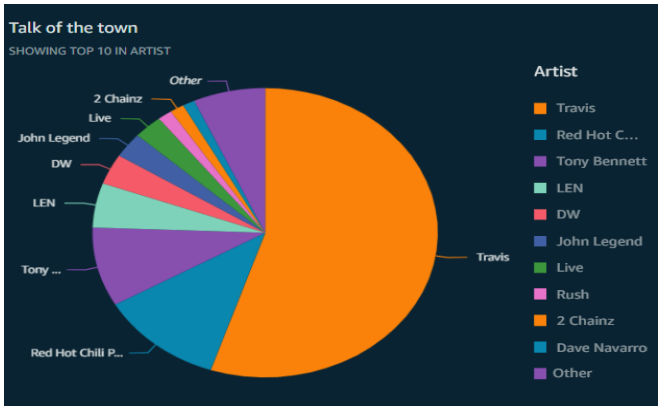
AWS Athena is a serverless interactive query service that makes it easy to analyze data directly in Amazon S3 using standard SQL. In this project, Athena is used to query the transformed data stored in S3, allowing for the extraction of meaningful insights without the need for complex ETL processes. Its serverless architecture ensures that you only pay for the queries you run, making it a cost-effective solution for large-scale data analysis. Athena's integration with S3 enables seamless querying of data, making it an ideal choice for this project's data analysis needs.




*This image shows a sample Athena query that is used to extract insights from the transformed data stored in S3. The query results are then visualized in the Amazon QuickSight dashboard.*

## Amazon QuickSight

Amazon QuickSight is a business intelligence service that allows users to create and publish interactive dashboards. In this project, QuickSight is employed to visualize the data extracted and processed from Reddit. With its ability to integrate seamlessly with AWS services, QuickSight provides a user-friendly interface for building dashboards that offer insights into music trends, artist popularity, and genre analysis. The interactive nature of QuickSight dashboards enables users to explore the data in a dynamic way, making it easier to identify patterns and correlations.

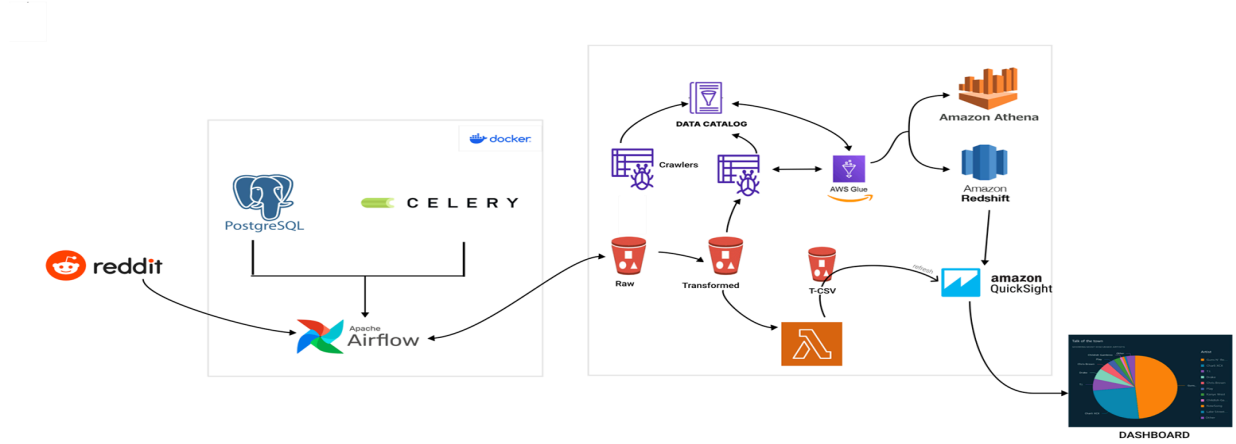



*This screenshot captures the Amazon QuickSight dashboard that visualizes the trending music topics and artists. It offers various interactive charts and graphs for deeper insights.*

# 3. Data Pipeline and Architecture

## Pipeline Overview

The data pipeline for the Reddit Music Trends Dashboard is designed to extract, process, and visualize data from the r/music subreddit in an automated and efficient manner. It begins with a Python script that leverages the Reddit API to extract top posts based on specific criteria. This data is then stored in Amazon S3, a scalable object storage service, in JSON format. AWS Glue, a fully managed ETL service, is employed to transform the raw data into a structured format suitable for analysis. AWS Athena, an interactive query service, is used to query the transformed data, and AWS Lambda orchestrates the various steps of the pipeline. Finally, Amazon QuickSight is used to create a dynamic, Billboard-like dashboard, providing visual insights into the music trends on Reddit.



*This image depicts the entire data pipeline architecture, from data extraction using Python scripts to final visualization in Amazon QuickSight. Each component in the pipeline is connected seamlessly to automate the data flow.*

## Step 1: Data Extraction

**Python Script with Reddit API:** The pipeline begins with script, crafted to connect with the Reddit platform, specifically targeting the r/music subreddit, and is responsible for extracting the top posts based on predefined criteria such as time and score. The extracted data includes crucial metadata like post titles, author information, and engagement metrics, which are essential for further analysis.

```python
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from pipelines.aws_s3_pipeline import upload_s3_pipeline
from pipelines.reddit_pipeline import reddit_pipeline

default_args = {
    'owner': 'Sneh Pillai',
    'start_date': datetime(2024, 7, 20)
}

file_postfix = datetime.now().strftime("%Y%m%d")

dag = DAG(
    dag_id='etl_reddit_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False,
    tags=['reddit', 'etl', 'pipeline']
)

# extraction from reddit
extract = PythonOperator(
    task_id='reddit_extraction',
    python_callable=reddit_pipeline,
    op_kwargs={
        'file_name': f'music_{file_postfix}',
        'subreddit': 'music',
        'time_filter': 'day',
        'limit': 100
    },
    dag=dag
)

# upload to s3
upload_s3 = PythonOperator(
    task_id='s3_upload',
    python_callable=upload_s3_pipeline,
    dag=dag
)

extract >> upload_s3
```

```python
def connect_reddit(client_id, client_secret, user_agent) -> Reddit:
    try:
        reddit = praw.Reddit(client_id=client_id,
                             client_secret=client_secret,
                             user_agent=user_agent)
        print("Connection to reddit successful")
        return reddit
    except Exception as e:
        print(e)
        sys.exit(1)


def extract_posts(reddit_instance: Reddit, subreddit: str, time_filter: str, limit=None):
    subreddit = reddit_instance.subreddit(subreddit)
    posts = subreddit.top(time_filter=time_filter, limit=limit)

    post_lists = []

    for post in posts:
        post_dict = vars(post)
        post = {key: post_dict[key] for key in POST_FIELDS}
        post_lists.append(post)

    return post_lists
```

*The Python script connected to the Reddit API is displayed in this screenshot, showing how it extracts the top posts from r/music. The script captures relevant fields such as post titles, upvotes, and artist mentions.*

## Step 2: Data Transformation

**AWS Glue:** Once the raw data is extracted, it is stored in an Amazon S3 bucket. AWS Glue is then employed to orchestrate the ETL (Extract, Transform, Load) process. Glue jobs clean and normalize the data, ensuring it is in a suitable format for analysis. For instance, posts are associated with artist names and genres, filling gaps where necessary and standardizing the data for consistent use across different stages of the pipeline.

**reddit_job_2**

Script | Job details | Runs | Data quality | Schedules | Version Control

**Script** Info

```
11
12  args = getResolvedOptions(sys.argv, ['JOB_NAME'])
13  sc = SparkContext()
14  glueContext = GlueContext(sc)
15  spark = glueContext.spark_session
16  job = Job(glueContext)
17  job.init(args['JOB_NAME'], args)
18
19  # Initialize boto3 S3 client
20  s3_client = boto3.client('s3')
21  bucket_name = 'redditpipeline-cis602'
22  prefix = 'raw/'
23
24  # List objects in the bucket
25  response = s3_client.list_objects_v2(Bucket=bucket_name, Prefix=prefix)
26  objects = response.get('Contents', [])
27
28  # Find the most recently created file
29  latest_file = max(objects, key=lambda x: x['LastModified'])
30  latest_file_key = latest_file['Key']
31
32  # Read the latest file into a DynamicFrame
33  AmazonS3_node = glueContext.create_dynamic_frame.from_options(
34      format_options={"quoteChar": "\"", "withHeader": True, "separator": ","},
35      connection_type="s3",
36      format="csv",
37      connection_options={"paths": [f"s3://{bucket_name}/{latest_file_key}"], "recurse": True},
```

**reddit_job_2**

Script | Job details | Runs | Data quality | Schedules | Version Control

**Script** Info

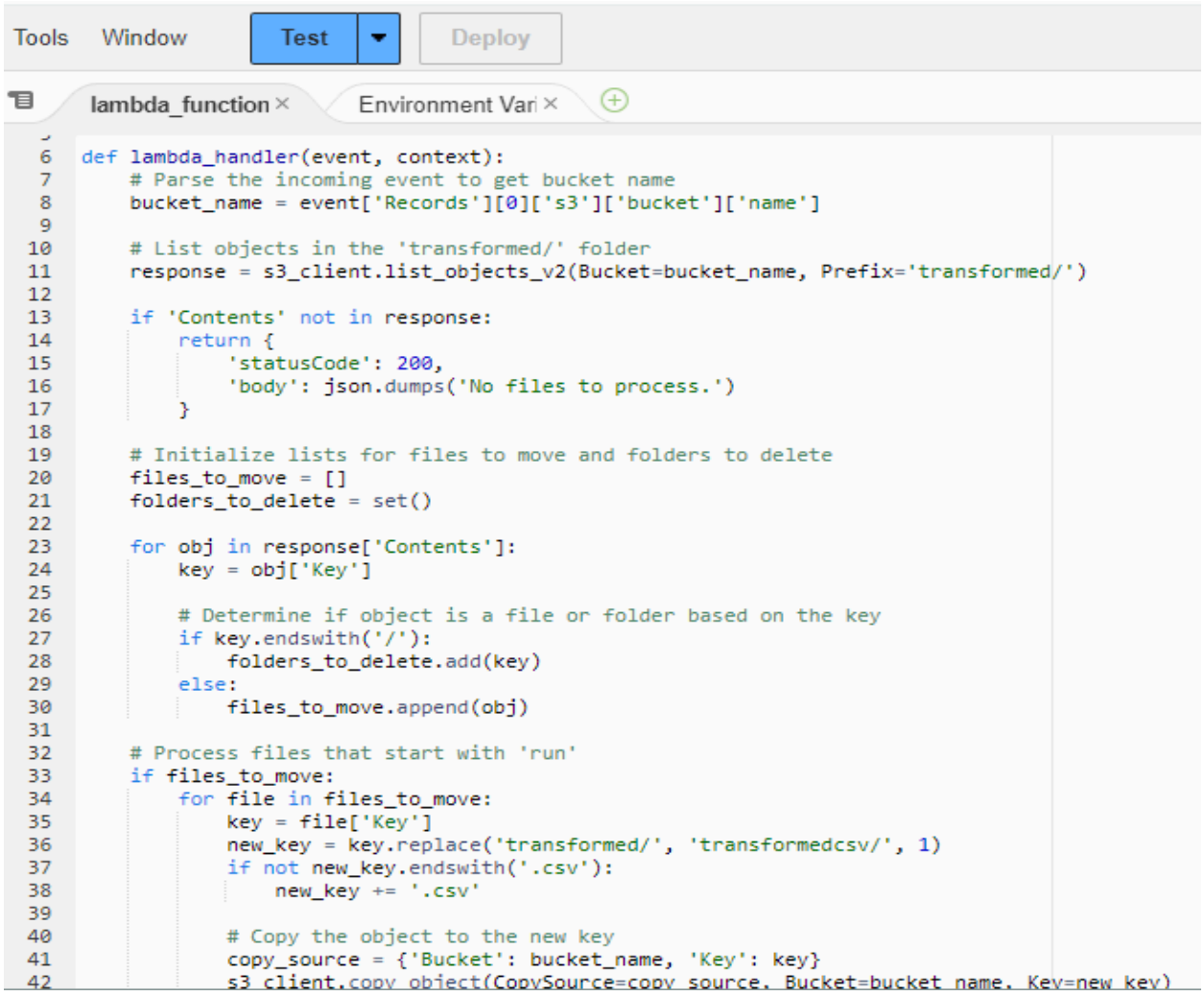
```
    transformation_ctx="AmazonS3_node"
)
# Convert DynamicFrame to DataFrame
df = AmazonS3_node.toDF()

# Concatenate columns
df_combined = df.withColumn('ESS_updated', concat_ws('-', df['edited'], df['spoiler'], df['stickied']))
df_combined = df_combined.drop('edited', 'spoiler', 'stickied')

# Replace empty strings with None
df_combined = df_combined.replace('', None)

# Fill empty artist and genre
df_combined = df_combined.fillna({'artist': 'Some New Artist', 'genre': 'Mix'})

# Convert back to DynamicFrame
S3bucket_node_combined = DynamicFrame.fromDF(df_combined, glueContext, 'S3bucket_node_combined')

# Write the transformed data to the destination S3 bucket
AmazonS3_node_combined = glueContext.write_dynamic_frame.from_options(
    frame=S3bucket_node_combined,
    connection_type="s3",
    format="csv",
    connection_options={"path": "s3://redditpipeline-cis602/transformed/", "partitionKeys": []},
    transformation_ctx="AmazonS3_node_combined"
)
job.commit()
```

*AWS Glue jobs responsible for transforming the raw Reddit data are highlighted in this screenshot. The transformation process includes data cleaning, parsing, and enriching the data with additional metadata like artist genres.*

## Step 3: Data Loading and Querying

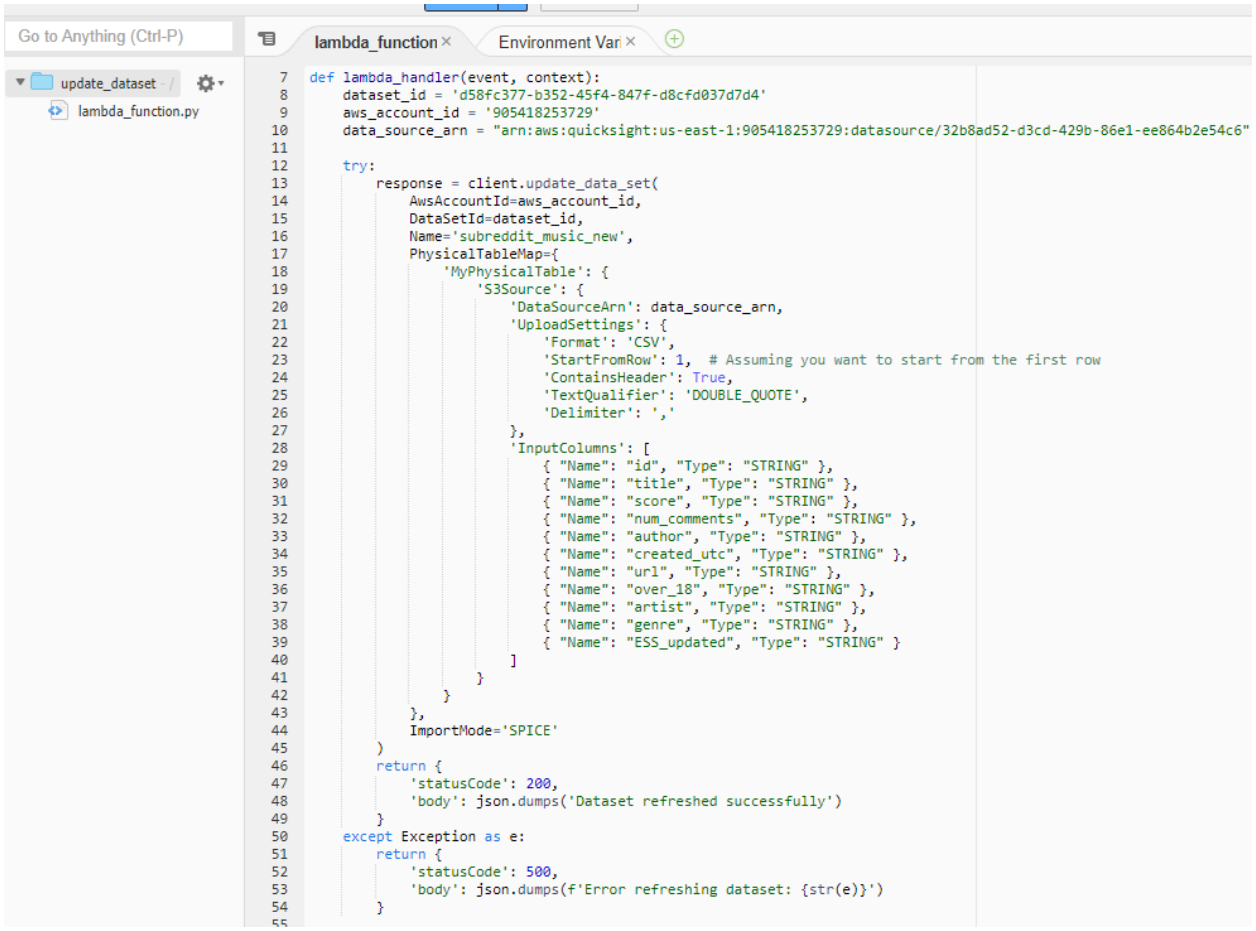
**AWS Lambda:** Post-transformation, the data is stored in a structured manner within another S3 bucket directory with the help of AWS Lambda functions we created. There are two Lambda functions we created which play a crucial role in processing and preparing the datasets for visualization in Amazon QuickSight.

1. Rename Dataset -
   The "Rename Dataset" Lambda function is designed to handle the issue where the output files from the AWS Glue job are saved without any file extensions. This function is triggered whenever a new file is placed into the "transformed" directory. Once triggered, the Lambda function renames the file by appending a `.csv` extension and then moves the file to the "transformedcsv" directory. This step is essential for ensuring that the file is recognized as a CSV, making it easier to process in subsequent steps.



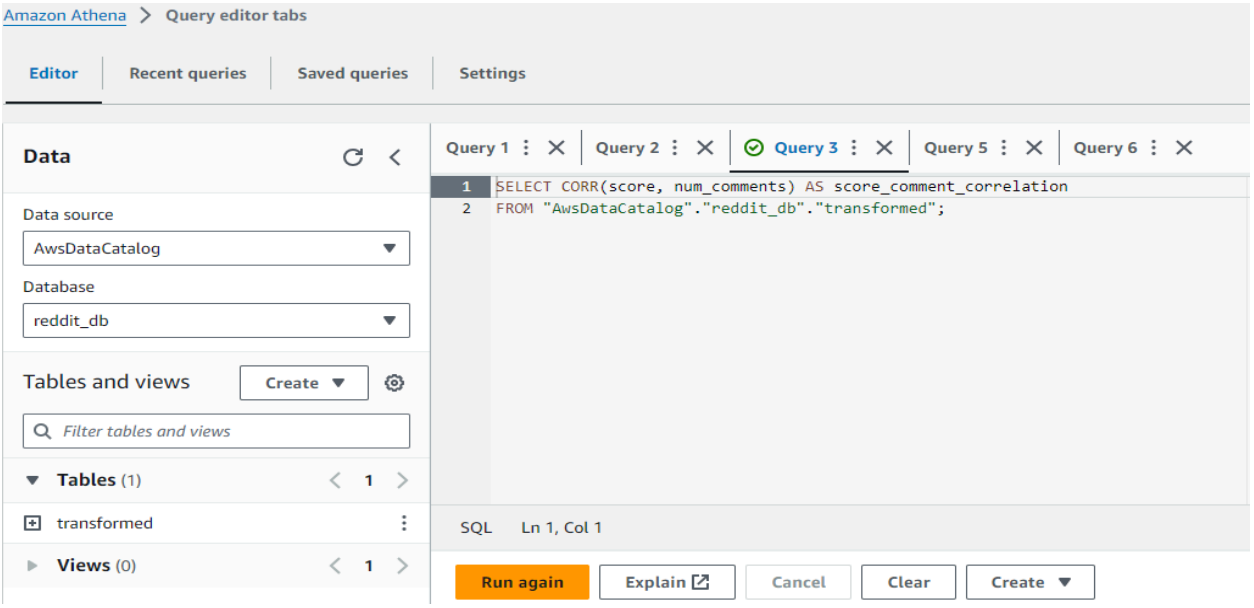
2. Update Dataset -

   The "Update Dataset" Lambda function is responsible for converting the newly formatted CSV file into a string format and then sending it to Amazon QuickSight for visualization. This function is triggered when a new file is added to the "transformedcsv" directory. The Lambda function reads the content of the CSV file, converts it into a string, and pushes it to QuickSight. This step is crucial for updating the dashboard in QuickSight with the latest data from the Reddit posts.




```python
def lambda_handler(event, context):
    dataset_id = 'd58fc377-b352-45f4-847f-d8cfd037d7d4'
    aws_account_id = '905418253729'
    data_source_arn = "arn:aws:quicksight:us-east-1:905418253729:datasource/32b8ad52-d3cd-429b-86e1-ee864b2e54c6"

    try:
        response = client.update_data_set(
            AwsAccountId=aws_account_id,
            DataSetId=dataset_id,
            Name='subreddit_music_new',
            PhysicalTableMap={
                'MyPhysicalTable': {
                    'S3Source': {
                        'DataSourceArn': data_source_arn,
                        'UploadSettings': {
                            'Format': 'CSV',
                            'StartFromRow': 1,  # Assuming you want to start from the first row
                            'ContainsHeader': True,
                            'TextQualifier': 'DOUBLE_QUOTE',
                            'Delimiter': ','
                        },
                        'InputColumns': [
                            { "Name": "id", "Type": "STRING" },
                            { "Name": "title", "Type": "STRING" },
                            { "Name": "score", "Type": "STRING" },
                            { "Name": "num_comments", "Type": "STRING" },
                            { "Name": "author", "Type": "STRING" },
                            { "Name": "created_utc", "Type": "STRING" },
                            { "Name": "url", "Type": "STRING" },
                            { "Name": "over_18", "Type": "STRING" },
                            { "Name": "artist", "Type": "STRING" },
                            { "Name": "genre", "Type": "STRING" },
                            { "Name": "ESS_updated", "Type": "STRING" }
                        ]
                    }
                }
            },
            ImportMode='SPICE'
        )
        return {
            'statusCode': 200,
            'body': json.dumps('Dataset refreshed successfully')
        }
    except Exception as e:
        return {
            'statusCode': 500,
            'body': json.dumps(f'Error refreshing dataset: {str(e)}')
        }
```

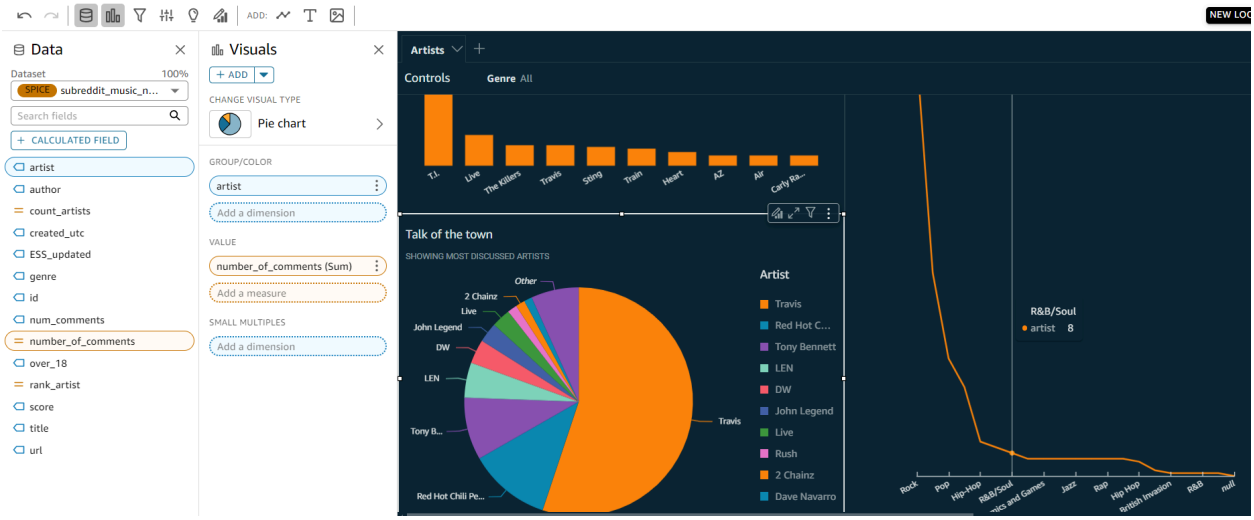
**AWS Athena:** In our pipeline, AWS Athena plays a crucial role in analyzing the data stored in Amazon S3. After the ETL (Extract, Transform, Load) process is completed by AWS Glue and the dataset is updated, AWS Athena is used to run SQL queries against the dataset. These queries are crucial for filtering, aggregating, and analyzing the data according to the specific requirements of the project. Athena's serverless architecture allows for efficient querying of large datasets directly from S3 without requiring additional infrastructure. This enables quick and cost-effective extraction of insights, making Athena an essential component in the pipeline for deriving meaningful information from the data.




*This image shows how AWS Lambda and Amazon Athena are used. The Lambda function triggers the data refresh in QuickSight, while Athena queries ensure that only the most relevant data is visualized.*

## Step 4: Data Visualization

**Amazon QuickSight:** The final, and perhaps most critical, component of the pipeline is the data visualization in Amazon QuickSight. Here, the processed data is transformed into insightful dashboards that highlight trends, correlations, and other key metrics related to the music industry. Users can explore trending artists, genres, and the overall sentiment in the subreddit with just a few clicks.



*Here, the final QuickSight dashboard is presented, showcasing the various visualizations generated from the processed Reddit data. The dashboard provides real-time insights into trending artists and music genres.*

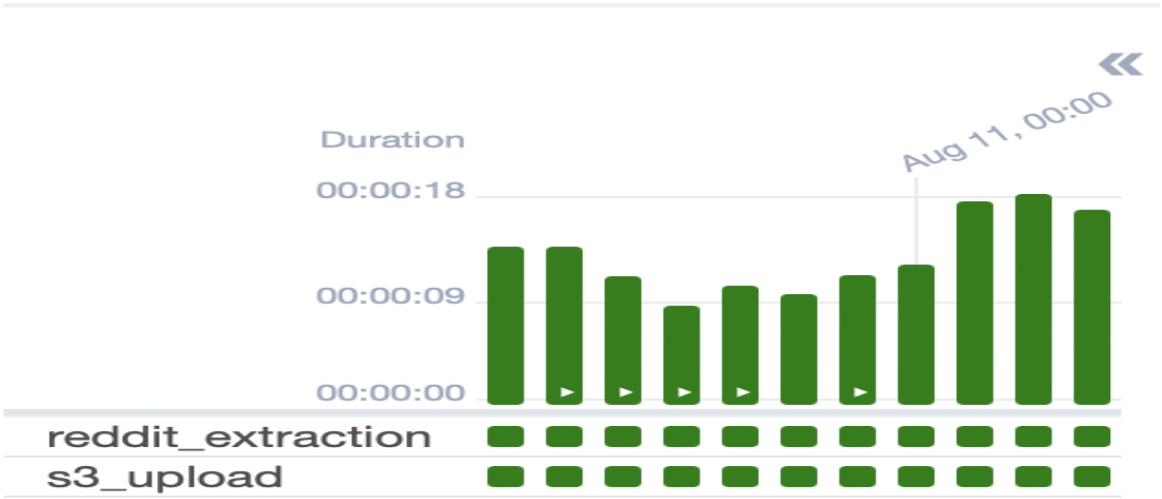
## Pipeline Orchestration

**Apache Airflow:** The orchestration of the data pipeline is managed by Apache Airflow, an open-source workflow management platform. Airflow coordinates the execution of various tasks, from data extraction to transformation and loading. The pipeline is defined as a Directed Acyclic Graph (DAG), where each task is represented as a node in the graph. The process starts with the extraction of top posts from the r/music subreddit using a Python script, followed by storing this data in Amazon S3. The subsequent steps involve triggering AWS Glue jobs for data transformation, executing AWS Lambda functions for data processing, and finally, loading the transformed data into Amazon QuickSight for visualization. Airflow's scheduling and monitoring capabilities ensure that the entire workflow runs smoothly and can be easily managed, retried, or scaled as needed.

requirements.txt M | docker-compose.yml M | reddit_dag.py 2 | merged_file.csv | config.conf | reddit_etl.py

dags > reddit_dag.py > ...

```python
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from pipelines.aws_s3_pipeline import upload_s3_pipeline
from pipelines.reddit_pipeline import reddit_pipeline

default_args = {
    'owner': 'Sneh Pillai',
    'start_date': datetime(2024, 7, 20)
}

file_postfix = datetime.now().strftime("%Y%m%d")

dag = DAG(
    dag_id='etl_reddit_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False,
    tags=['reddit', 'etl', 'pipeline']
)

# extraction from reddit
extract = PythonOperator(
    task_id='reddit_extraction',
    python_callable=reddit_pipeline,
    op_kwargs={
        'file_name': f'music_{file_postfix}',
        'subreddit': 'music',
        'time_filter': 'day',
        'limit': 100
    },
    dag=dag
)

# upload to s3
upload_s3 = PythonOperator(
    task_id='s3_upload',
    python_callable=upload_s3_pipeline,
    dag=dag
)

extract >> upload_s3
```



*The Apache Airflow DAG, visible in this screenshot, monitors the entire pipeline execution. Each task in the DAG is crucial for maintaining the data pipeline's integrity and ensuring that data updates are processed in the correct order.*

# 4. Conclusion

In this project, we successfully designed and implemented a comprehensive data pipeline that transforms Reddit data into actionable insights through an intuitive and interactive Amazon QuickSight dashboard. By utilizing a combination of AWS services such as S3, Glue, Lambda, and Athena, we ensured that the data pipeline is robust, scalable, and capable of handling real-time updates.

The project's significance lies in its ability to provide real-time analysis of music trends, helping industry professionals, artists, and enthusiasts stay informed about the latest developments in the music world. The automation facilitated by Apache Airflow ensures that the pipeline remains efficient and requires minimal manual intervention.

Looking ahead, the project has the potential for several enhancements. Expanding the data sources to include other platforms like Twitter or YouTube, integrating sentiment analysis to gauge public opinion, and optimizing the dashboard for more detailed analytics are just a few possibilities. These improvements would not only enhance the richness of the insights but also broaden the scope of the dashboard to serve a wider audience.

The successful completion of this project demonstrates the effectiveness of modern data engineering techniques in leveraging social media data for industry insights. It highlights the potential for further innovation in this space, paving the way for more sophisticated data-driven solutions in the music industry and beyond.